

Ассимиляция данных и физически-информированные нейросети. Введение

Data assimilation (DA) techniques

Начало методов ассимиляции данных (data assimilation techniques) было положено в 50-х гг. прошлого века¹, когда данные температуры со спутников смогли ассимилировать с помощью численных моделей для предсказания погоды. Одна из типичных реализаций предполагает определение начального поля на основе разреженных и ограниченных по времени измерений, после чего выполняется численное прогнозирование будущих состояний.

DA предоставляет мощный инструмент, объединяющий преимущества **экспериментальных измерений и численного моделирования**. Благодаря этому он играет важную роль как в расширении возможностей экспериментального анализа, так и в количественной оценке неопределённости численных моделей.

¹Charney J., Halem M., Jastrow R. Use of incomplete historical data to infer the present state of the atmosphere. // Journal of Atmospheric Sciences. 1969. Т. 26, № 5. С. 1160—1163.

Концепция DA

Наблюдения (экспериментальные измерения)



Алгоритмы ассимиляции данных (байесовские, вариационные
методы, ML)



Прогностическая модель (физические ограничения)



Прогноз будущих состояний системы

Методы ассимиляции данных

Методы	Детали	Задачи	Сложность
Подстановка	Крупномасштабная подстановка	Синхронизация турбулентности для DNS	★
	Подстановка ГУ и НУ	Мгновенное восстановление поля течения	★
	Подстановка поля течения	Мгновенное определение давления	★★
Оптимальная аппроксимация	Кригинг	Моделирование пониженного порядка	★★
	RBF (интерполяция с помощью радиально-базисных функций)	Мгновенная интерполяция поля течения	★★★
	FlowFit	Интерполяция поля течения на основе 3D-LPT	★★★
	VIC+/VIC#	Интерполяция с учётом временной информации	★★★ $\frac{1}{2}$
Ансамблевые модели	EnKF (ensemble Kalman filter)	Восстановление среднего поля; нестационарное интегрирование	★★★
Вариационные методы	Continuous 3DVar	Восстановление среднего поля течения	★★
	Discrete 3DVar	Восстановление среднего поля течения	★★★
	Continuous 4DVar	Восстановление нестационарного поля течения	★★★★
	Discrete 4DVar	Восстановление нестационарного поля течения	★★★★★
Ensemble variation	EnVar	Восстановление среднего поля; нестационарное интегрирование	★★★★
Машинное обучение	PINNs	Восстановление среднего поля; мгновенная интерполяция	★★★★★

KF[2; 3]. Постановка задачи

Рассматривается объект, движущийся вдоль одной координаты.

Истинное положение объекта неизвестно и изменяется по закону

$$x_{k+1} = x_k + u_k + \xi_k,$$

где u_k — известное управляющее воздействие, а ξ_k — случайная ошибка модели с дисперсией σ_ξ^2 .

Положение объекта измеряется сенсором:

$$z_k = x_k + \eta_k,$$

где η_k — случайная ошибка измерения с дисперсией σ_η^2 .

Требуется:

- оценить истинное положение x_k по шумным измерениям z_k ;
- учесть известное управление u_k ;
- объединить предсказание модели и данные сенсора;
- минимизировать средний квадрат ошибки оценки.

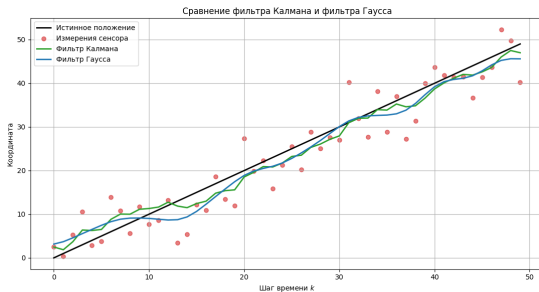
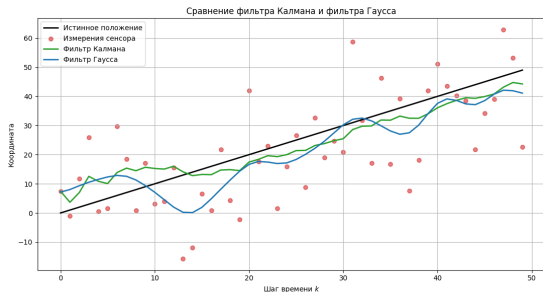


Рис. 1 – $\sigma_{sensor}^1 = 15$, $\sigma_{sensor}^2 = 5$

Одномерный фильтр Калмана

Пусть на шаге k известно оптимальное значение x_k^{opt} . Тогда предсказание координаты на следующем шаге равно

$$\hat{x}_{k+1} = x_k^{\text{opt}} + u_k.$$

Новое значение объединяется с предсказанием:

$$x_{k+1}^{\text{opt}} = K_{k+1} z_{k+1} + (1 - K_{k+1}) (x_k^{\text{opt}} + u_k), \quad x_0^{\text{opt}} = z_0.$$

Коэффициент K_{k+1} выбирается так, чтобы минимизировать средний квадрат ошибки:

$$\mathbb{E}(e_{k+1}^2) = \mathbb{E} \left[\left(x_{k+1}^{\text{opt}} - x_{k+1} \right)^2 \right].$$

С учётом независимости ошибок модели и сенсора:

$$\mathbb{E}(e_{k+1}^2) = \frac{\sigma_\eta^2 \left[\mathbb{E}(e_k^2) + \sigma_\xi^2 \right]}{\mathbb{E}(e_k^2) + \sigma_\xi^2 + \sigma_\eta^2}.$$

Следовательно, коэффициент Калмана равен

$$K_{k+1} = \frac{\mathbb{E}(e_k^2) + \sigma_\xi^2}{\mathbb{E}(e_k^2) + \sigma_\xi^2 + \sigma_\eta^2} = \frac{\mathbb{E}(e_{k+1}^2)}{\sigma_\eta^2}.$$

Обозначения

u_k — известная управляющая функция, ξ_k — ошибка модели, $\text{Var}(\xi_k) = \sigma_\xi^2$,

η_k — ошибка сенсора, $\text{Var}(\eta_k) = \sigma_\eta^2$.

Модель движения и измерений:

$$x_{k+1} = x_k + u_k + \xi_k, \quad z_k = x_k + \eta_k.$$

Ошибка оценки и коэффициент Калмана:

$$E(e_0^2) = E(\eta_0^2) = \sigma_\eta^2,$$

$$E(e_{k+1}^2) = \frac{\sigma_\eta^2 (E(e_k^2) + \sigma_\xi^2)}{E(e_k^2) + \sigma_\xi^2 + \sigma_\eta^2},$$

$$K_{k+1} = \frac{E(e_k^2) + \sigma_\xi^2}{E(e_k^2) + \sigma_\xi^2 + \sigma_\eta^2} = \frac{E(e_{k+1}^2)}{\sigma_\eta^2}.$$

Оптимальная оценка:

$$x_{k+1}^{\text{opt}} = K_{k+1} z_{k+1} + (1 - K_{k+1}) (x_k^{\text{opt}} + u_k), \quad x_0^{\text{opt}} = z_0.$$

Ансамблевый фильтр Калмана

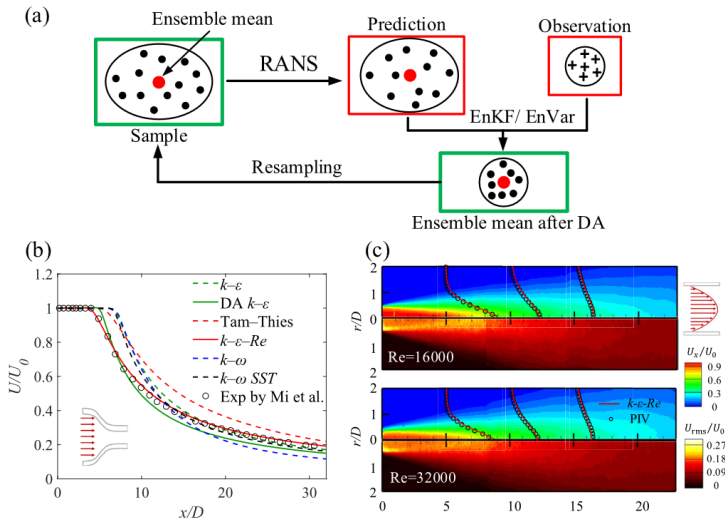


Рис. 2 – (a) Отдельная точка – состояние системы, например, поле скорости.[4]

Ансамблевый фильтр Калмана

Вместо одной оценки состояния используется ансамбль из N возможных состояний:

$$\left\{ \mathbf{x}_k^{(1)}, \mathbf{x}_k^{(2)}, \dots, \mathbf{x}_k^{(N)} \right\}.$$

1. Инициализация

$$\mathbf{x}_0^{(i)} \sim p(\mathbf{x}_0), \quad i = 1, \dots, N.$$

2. Прогнозирование

Каждый элемент ансамбля пропускается через модель:

$$\mathbf{x}_{k+1}^{(i)} = f\left(\mathbf{x}_k^{(i)}, \mathbf{u}_k\right) + \mathbf{w}_k^{(i)}.$$

3. Оценка статистик ансамбля

$$\bar{\mathbf{x}}_{k+1} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{k+1}^{(i)},$$

$$\mathbf{P}_{k+1} \approx \frac{1}{N-1} \sum_{i=1}^N \left(\mathbf{x}_{k+1}^{(i)} - \bar{\mathbf{x}}_{k+1} \right) \left(\mathbf{x}_{k+1}^{(i)} - \bar{\mathbf{x}}_{k+1} \right)^T.$$

4. Коррекция по измерению

$$\mathbf{K}_{k+1} = \mathbf{P}_{k+1} \mathbf{H}^T \left(\mathbf{H} \mathbf{P}_{k+1} \mathbf{H}^T + \mathbf{R} \right)^{-1},$$

$$\mathbf{x}_{k+1}^{(i)} \leftarrow \mathbf{x}_{k+1}^{(i)} + \mathbf{K}_{k+1} \left(\mathbf{z}_{k+1}^{(i)} - \mathbf{H} \mathbf{x}_{k+1}^{(i)} \right).$$

Обозначения в ансамблевом фильтре Калмана

- $\mathbf{x}_k$ — вектор состояния системы в момент времени k ;
- $\mathbf{x}_k^{(i)}$ — i -й элемент ансамбля, то есть одна возможная реализация состояния системы;
- N — количество элементов ансамбля;
- $\mathbf{u}_k$ — вектор управляющего воздействия;
- $f(\cdot)$ — функция математической модели системы;
- $\mathbf{w}_k^{(i)}$ — случайная ошибка модели для i -го элемента ансамбля;
- $\mathbf{z}_k$ — вектор измерений сенсора;
- $\mathbf{z}_k^{(i)}$ — возмущённое измерение, используемое для i -го элемента ансамбля;
- $\mathbf{H}$ — матрица наблюдения, связывающая состояние системы с измерениями:

$$\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k;$$

- $\mathbf{v}_k$ — ошибка измерения сенсора;
- $\mathbf{R}$ — ковариационная матрица ошибки измерения;
- $\bar{\mathbf{x}}_k$ — среднее значение ансамбля, используемое как оценка состояния;
- $\mathbf{P}_k$ — ковариационная матрица состояния, оценённая по ансамблю;
- $\mathbf{K}_k$ — коэффициент усиления Калмана.

Вариационный метод

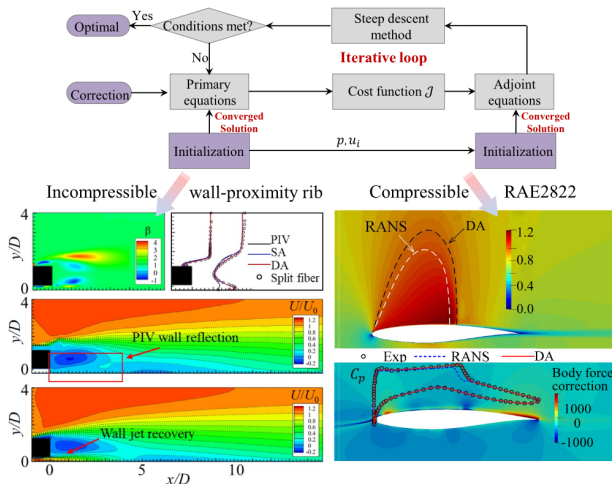


Рис. 3 – Вариационная ассимиляция данных для обратного восстановления формы с использованием сопряжённых уравнений. ВМ превращает задачу ассимиляции данных в задачу минимизации функционала, где ошибка между моделью и наблюдениями уменьшается с учётом уравнений движения, априорной информации и ограничений на решение. [4]

Сравнение фильтра Калмана и вариационных методов

Свойство	Фильтр Калмана	Вариационный метод
Основной подход	Последовательное обновление	Минимизация функционала
Используемые данные	Текущее измерение	Набор измерений за интервал
Результат	Оценка состояния в текущий момент	Оптимальное состояние или траектория
Вычислительный метод	Аналитические формулы	Градиентная оптимизация
Обработка нелинейности	EKF, UKF, EnKF	3D-Var, 4D-Var
Работа в реальном времени	Обычно удобна	Может быть вычислительно затратной

PINN

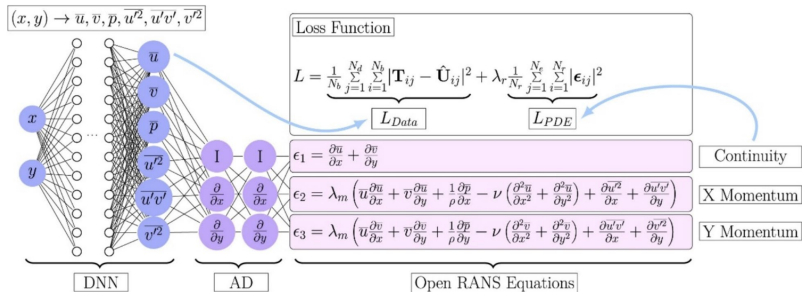


Рис. 4 – PINN [4]

Оригинальная статья о PINN

Оригинальная статья Raissi² [git репозиторий](#)

Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations

[M Raissi](#), [P Perdikaris](#), [GE Karniadakis](#) - Journal of Computational **physics** **2019** Elsevier

... We introduce **physics-informed neural networks** – **neural networks** that are trained to solve supervised learning tasks while respecting any given laws of **physics** described by general ...

☆ Save [Cite](#) **Cited by 13291** [Related articles](#) [All 8 versions](#)

2025 г.

Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations

[M Raissi](#), [P Perdikaris](#), [GE Karniadakis](#) - Journal of Computational **physics**, 2019 - Elsevier

... We introduce **physics-informed neural networks** – **neural networks** that are trained to solve supervised learning tasks while respecting any given laws of **physics** described by general ...

☆ Save [Cite](#) **Cited by 21962** [Related articles](#) [All 9 versions](#)

03.2026

09.2026 – 28723 цитирований.

²*Raissi M., Perdikaris P., Karniadakis G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. // Journal of Computational physics. 2019. T. 378. С. 686—707.*

...the proposed work aims to go one step further by revisiting the construction of “custom” activation and loss functions that are tailored to the underlying differential operator. This allows us **to open the black-box** by understanding and appreciating the key role played by **automatic differentiation** within the deep learning field. Automatic differentiation in general, and **the back-propagation algorithm** in particular, is currently the dominant approach for training deep models by taking their derivatives with respect to the parameters (e.g., weights and biases) of the models.[5]

PINN

2 класса задач



data-driven solution

- Аппроксиматоры пространственно-временных функций с использованием данных
- Решение задач подобно численному моделированию

data-driven discovery of partial differential equations

- Восстановление коэффициентов уравнений при подстановке синтетических или экспериментальных данных в уравнения
- Решение обратных задач

Три вида дифференцирования[6]

Рассмотрим функцию

$$f(x) = x^2 \sin x.$$

Численное дифференцирование

Производная приближённо вычисляется по конечным разностям:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad h \rightarrow 0.$$

Результат — численное приближение производной.

Символьное дифференцирование

Математическое выражение преобразуется в явную формулу:

$$\frac{d}{dx} (x^2 \sin x) = 2x \sin x + x^2 \cos x.$$

Результат — аналитическое выражение производной.

Автоматическое дифференцирование

Производная вычисляется автоматически для последовательности элементарных операций с использованием цепного правила:

$$u = x^2, \quad v = \sin x, \quad f = uv,$$

$$\frac{df}{dx} = \frac{du}{dx} v + u \frac{dv}{dx} = 2x \sin x + x^2 \cos x.$$

Результат — численное значение производной.

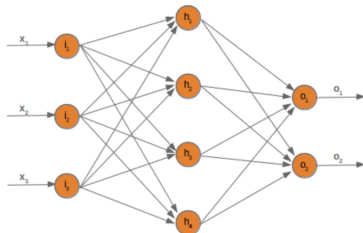


Рис. 5 – Прямой проход (forward propagation)

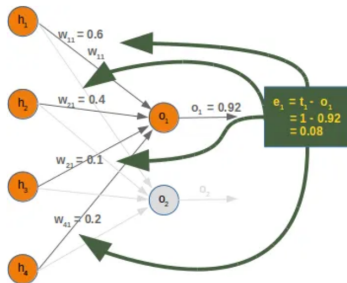


Рис. 6 – Обратный проход (backward propagation)

Алгоритм обучения

Для каждого d из набора данных выполнить:

Прямой проход

Начиная с входного слоя, используя уравнение (1), выполнить прямой проход через сеть.

Обратное распространение

Вычислить производные функции ошибки по активациям выходного слоя.

Для каждого слоя l от выходного до второго выполнить:

- вычислить производные функции ошибки по входам нейронов верхнего слоя;
- вычислить производные функции ошибки по весам между внешним слоем и расположенным ниже слоем;
- вычислить производные функции ошибки по активациям расположенного ниже слоя.

Конец цикла.

Обновить веса.

Конец цикла.

Что понадобится для работы

- google colab
<https://colab.research.google.com>
- linux или WSL (Windows Subsystem for Linux)
- git
<https://www.w3schools.com/git/default.asp?remote=github>
- jupyter notebook
<https://jupyter.org/install>

1. *Charney J., Halem M., Jastrow R.* — Use of incomplete historical data to infer the present state of the atmosphere. — // *Journal of Atmospheric Sciences.* — 1969. — T. 26, № 5. — С. 1160—1163.
2. URL: <https://habr.com/ru/articles/166693/>.
3. URL: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python>.
4. *He C., Li S., Liu Y.* — Data assimilation: new impetus in experimental fluid dynamics. — // *Experiments in Fluids.* — 2025. — T. 66, № 5. — С. 94.
5. *Raissi M., Perdikaris P., Karniadakis G. E.* — Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. — // *Journal of Computational physics.* — 2019. — T. 378. — С. 686—707.

6. URL: `https://huggingface.co/blog/andmholm/what-is-automatic-differentiation`.